Java Inheritance

Source code, explanation, and verified sample output

Objective

This program demonstrates the major inheritance structures used in Java. It uses classes for single, multilevel, and hierarchical inheritance, interfaces to model multiple inheritance, and a class-plus-interface design for hybrid inheritance.

Inheritance Types Demonstrated

Single inheritance

Dog extends Animal. Dog receives the name field and eat() method from one parent class.

Multilevel inheritance

Puppy extends Dog, and Dog extends Animal. Puppy can use members from both earlier levels.

Hierarchical inheritance

Dog and Cat each extend Animal. Several child classes share one parent class.

Multiple inheritance

Java disallows extending more than one class. SmartPhone implements Telephone and Camera interfaces to combine two capabilities safely.

Hybrid inheritance

ElectricCar extends Vehicle and implements Rechargeable, combining class inheritance with interface inheritance.

Complete Java Source Code

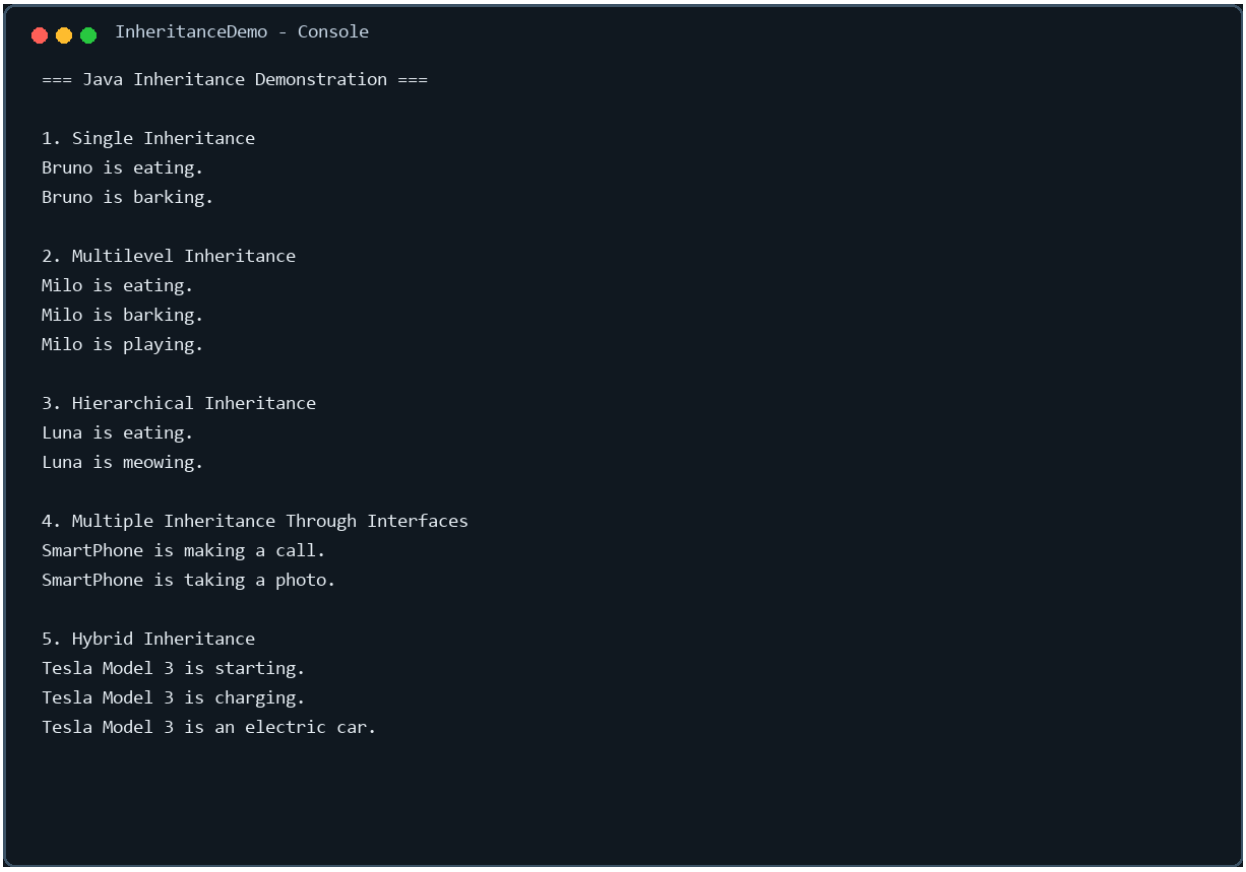
```
/**
 * Demonstrates the principal forms of inheritance supported or modeled in Java.
 * Java does not permit multiple inheritance of classes, so interfaces are used
 * to safely provide the multiple-inheritance behavior.
 */
public class InheritanceDemo {
    public static void main(String[] args) {
        System.out.println("=== Java Inheritance Demonstration ===\n");

        System.out.println("1. Single Inheritance");
        Dog dog = new Dog("Bruno");
        dog.eat();                // inherited from Animal
        dog.bark();               // defined in Dog

        System.out.println("\n2. Multilevel Inheritance");
        Puppy puppy = new Puppy("Milo");
        puppy.eat();              // inherited through Dog from Animal
        puppy.bark();             // inherited from Dog
        puppy.play();             // defined in Puppy

        System.out.println("\n3. Hierarchical Inheritance");
        Cat cat = new Cat("Luna");
        cat.eat();                // inherited from Animal
        cat.meow();               // defined in Cat

        System.out.println("\n4. Multiple Inheritance Through Interfaces");
        SmartPhone phone = new SmartPhone();
        phone.call();             // from Telephone interface
        phone.takePhoto();        // from Camera interface

        System.out.println("\n5. Hybrid Inheritance");
        ElectricCar electricCar = new ElectricCar("Tesla Model 3");
        electricCar.start();      // inherited from Vehicle
        electricCar.charge();     // from Rechargeable interface
        electricCar.showType();   // defined in ElectricCar
    }
}

// Parent class used for single, multilevel, and hierarchical inheritance.
class Animal {
    protected String name;

    Animal(String name) {
        this.name = name;
    }

    void eat() {
        System.out.println(name + " is eating.");
    }
}

// SINGLE INHERITANCE: Dog inherits members of one parent class, Animal.
class Dog extends Animal {
    Dog(String name) {
        super(name);
    }

    void bark() {
        System.out.println(name + " is barking.");
    }
}

// MULTILEVEL INHERITANCE: Puppy inherits from Dog, which already inherits Animal.
class Puppy extends Dog {
    Puppy(String name) {
        super(name);
    }

    void play() {
        System.out.println(name + " is playing.");
    }
}

// HIERARCHICAL INHERITANCE: Cat and Dog are different children of Animal.
class Cat extends Animal {
    Cat(String name) {
        super(name);
    }

    void meow() {
        System.out.println(name + " is meowing.");
    }
}

// Interfaces declare independent capabilities, not shared class state.
interface Telephone {
    void call();
```

```
}

interface Camera {
    void takePhoto();
}

// MULTIPLE INHERITANCE (via interfaces): one class implements both capabilities.
class SmartPhone implements Telephone, Camera {
    public void call() {
        System.out.println("SmartPhone is making a call.");
    }

    public void takePhoto() {
        System.out.println("SmartPhone is taking a photo.");
    }
}

class Vehicle {
    protected String model;

    Vehicle(String model) {
        this.model = model;
    }

    void start() {
        System.out.println(model + " is starting.");
    }
}

interface Rechargeable {
    void charge();
}

// HYBRID INHERITANCE: ElectricCar extends Vehicle and implements Rechargeable.
class ElectricCar extends Vehicle implements Rechargeable {
    ElectricCar(String model) {
        super(model);
    }

    public void charge() {
        System.out.println(model + " is charging.");
    }

    void showType() {
        System.out.println(model + " is an electric car.");
    }
}
```

Execution and Test Results

The program was compiled and executed with the following test objects: Dog (Bruno), Puppy (Milo), Cat (Luna), SmartPhone, and ElectricCar (Tesla Model 3). The console output verifies that inherited and interface-provided methods are accessible in every case.

Console Output Screenshot



How to Run

Save the source as InheritanceDemo.java, compile with `javac InheritanceDemo.java`, and run with `java InheritanceDemo`.